

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM	:	123240232 123240254
NAMA	:	Azarya Shalom Widura Ajrul ramadhan
PLUG	:	IF- G
NAMA ASISTEN	:	Farhannivta Ramadhana Muhammad Ruhul Jadid

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Azarya Shalom Widura 123240232

Ajrun Ramadhan 123240254

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung

Keputusan

Pada Tanggal:

Asisten Praktikum

Asisten Praktikum

Farhannivta Ramadhana

NIM. 123230139

Muhammad Ruhul Jadid

NIM. 123230046

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul *SPK Pemilihan Smartwatch menggunakan metode AHP*. Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 28 Mei 2026

Penyusun 1,

Penyusun 2,

Azarya Shalom Widura

NIM. 123240232

Ajrun Ramadhan

NIM. 123240254

DAFTAR ISI

HALAMAN PENGESAHAN	ii
KATA PENGANTAR.....	iii
DAFTAR ISI	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Tujuan Proyek Akhir.....	2
1.3 Manfaat Proyek Akhir.....	2
BAB II PEMBAHASAN.....	3
2.1 Dasar Teori	3
2.1.1 Sistem Pendukung Keputusan.....	3
2.1.2 Analytical Hierarchy Process (AHP).....	3
2.1.3 Rumus-Rumus AHP	4
2.2 Skenario Kasus & Dataset	5
2.3 Pemodelan Sistem Pendukung keputusan.....	6
2.3.1. Kriteria Penilaian	6
2.3.2. Alternatif	6
2.3.3. Skala Penilaian Perbandingan Pasangan	7
2.4 Perhitungan & Algoritma	7
2.4.1. Matriks Perbandingan Berpasangan.....	7
2.4.2. Normalisasi Matriks dan Perhitungan Bobot	8
2.4.3. Perhitungan Consistency Ratio (CR)	9
2.4.4. Algoritma Perangkingan Alternatif	9
2.5 Implementasi & Tampilan Sistem	10
BAB III JADWAL Pengerjaan dan Pembagian Tugas	48
3.1 Jadwal Pengerjaan	48
3.2 Pembagian Tugas.....	48
BAB IV KESIMPULAN DAN SARAN.....	49
4.2 Kesimpulan	49
4.3 Saran	49
DAFTAR PUSTAKA.....	51

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan teknologi wearable device, khususnya smartwatch, telah mengalami kemajuan yang sangat pesat dalam beberapa tahun terakhir. Berbagai merek ternama seperti Apple, Samsung, Garmin, Fitbit, Huawei, Fossil, Xiaomi, dan Polar berlomba-lomba menghadirkan produk dengan fitur-fitur unggulan yang beragam. Kondisi ini menciptakan pasar yang kompetitif sekaligus membingungkan bagi konsumen yang ingin memilih smartwatch terbaik sesuai kebutuhan dan anggaran.

Proses pemilihan smartwatch melibatkan berbagai kriteria yang saling terkait, antara lain harga, ketahanan baterai, kemampuan ketahanan air, ketersediaan sensor detak jantung (heart rate monitor), dan fitur GPS. Ketika konsumen harus mempertimbangkan banyak kriteria secara bersamaan, pengambilan keputusan yang bersifat intuitif cenderung menghasilkan pilihan yang kurang optimal dan tidak dapat dipertanggungjawabkan secara sistematis.

Berdasarkan permasalahan tersebut, diperlukan sebuah Sistem Pendukung Keputusan (SPK) yang mampu memproses berbagai kriteria secara terstruktur dan menghasilkan rekomendasi yang objektif. Metode *Analytical Hierarchy Process* (AHP) yang diperkenalkan oleh Thomas L. Saaty pada tahun 1980 dipilih sebagai pendekatan dalam proyek akhir ini. AHP memungkinkan pengambil keputusan untuk membandingkan tingkat kepentingan antar kriteria secara berpasangan, kemudian menghitung bobot masing-masing kriteria secara konsisten dan terverifikasi melalui nilai Consistency Ratio (CR).

Sistem ini dibangun menggunakan bahasa pemrograman Python dengan framework Streamlit, sehingga menghasilkan aplikasi web interaktif yang memudahkan pengguna dalam melakukan eksplorasi data, manajemen data produk, serta melihat hasil perankingan rekomendasi smartwatch secara real-time.

1.2 Tujuan Proyek Akhir

1. Membangun Sistem Pendukung Keputusan (SPK) berbasis web yang dapat membantu pengguna dalam proses pemilihan smartwatch.
2. Mengimplementasikan metode Analytical Hierarchy Process (AHP) sebagai dasar pembobotan kriteria dan perankingan alternatif produk smartwatch.
3. Menyediakan fitur validasi konsistensi penilaian melalui perhitungan Consistency Ratio (CR) sehingga bobot yang dihasilkan dapat dipercaya.
4. Menyediakan antarmuka pengguna yang interaktif dan intuitif untuk eksplorasi data, manajemen data, pembobotan kriteria, serta visualisasi hasil rekomendasi.

1.3 Manfaat Proyek Akhir

1. Bagi Konsumen: Memudahkan proses pengambilan keputusan pemilihan smartwatch yang lebih objektif, sistematis, dan dapat dipertanggungjawabkan berdasarkan preferensi pribadi.
2. Bagi Akademis: Memberikan contoh penerapan nyata metode AHP dalam pengembangan Sistem Pendukung Keputusan di bidang e-commerce dan pemilihan produk teknologi.
3. Bagi Pengembang: Menjadi referensi dalam membangun sistem rekomendasi berbasis multi-kriteria menggunakan Python dan Streamlit.

BAB II

PEMBAHASAN

2.1 Dasar Teori

2.1.1 Sistem Pendukung Keputusan

Sistem Pendukung Keputusan (SPK) atau *Decision Support System* (DSS) merupakan sistem informasi berbasis komputer yang dirancang untuk membantu pengambil keputusan dalam mengolah data dan informasi sehingga dapat menghasilkan keputusan yang lebih baik. SPK tidak menggantikan peran pengambil keputusan, melainkan memberikan dukungan melalui analisis data yang sistematis dan terstruktur (Turban et al., 2005).

Komponen utama SPK terdiri atas tiga bagian, yaitu manajemen data (database), manajemen model (berisi metode dan algoritma pengambilan keputusan), serta antarmuka pengguna yang memfasilitasi interaksi antara sistem dan pengguna.

2.1.2 Analytical Hierarchy Process (AHP)

Analytical Hierarchy Process (AHP) adalah metode pengambilan keputusan multi-kriteria yang dikembangkan oleh Thomas L. Saaty pada tahun 1980. AHP bekerja dengan cara memecah masalah yang kompleks menjadi hierarki yang terdiri atas tujuan, kriteria, dan alternatif, kemudian melakukan perbandingan berpasangan (*pairwise comparison*) antar elemen pada setiap tingkatan hierarki (Saaty, 1980).

Langkah-langkah utama dalam metode AHP adalah sebagai berikut:

1. **Penyusunan Matriks Perbandingan Berpasangan:** Setiap kriteria dibandingkan satu sama lain menggunakan skala Saaty (1–9) untuk menentukan tingkat kepentingan relatifnya.
2. **Normalisasi Matriks:** Setiap elemen matriks dibagi dengan jumlah total kolom yang bersangkutan untuk mendapatkan matriks ternormalisasi.
3. **Perhitungan Bobot Prioritas:** Bobot prioritas diperoleh dengan menghitung rata-rata setiap baris matriks ternormalisasi.

4. **Perhitungan Consistency Ratio (CR):** Konsistensi penilaian diuji melalui perhitungan CI dan CR. Penilaian diterima jika $CR \leq 0,10$.

Skala perbandingan berpasangan yang digunakan dalam metode AHP adalah Skala Saaty, sebagaimana disajikan pada Tabel 2.1 berikut.

Tabel 2.1 Skala Perbandingan Berpasangan (Skala Saaty)

Skala	Definisi Verbal	Keterangan
1	Sama pentingnya (Equal Importance)	Kedua elemen memiliki pengaruh yang setara
2	Antara sama penting dan sedikit lebih penting	Diperlukan pertimbangan lebih lanjut
3	Sedikit lebih penting (Moderate Importance)	Penilaian dan pengalaman sedikit mendukung satu elemen
4	Antara sedikit dan cukup lebih penting	Penilaian dan pengalaman kuat mendukung satu elemen
5	Cukup lebih penting (Strong Importance)	Satu elemen secara kuat lebih disukai
6	Antara cukup dan sangat lebih penting	Preferensi sangat kuat, didominasi secara praktis
7	Sangat lebih penting (Very Strong Importance)	Satu elemen sangat mendominasi dalam praktik
8	Antara sangat dan mutlak lebih penting	Mendekati kepentingan mutlak
9	Mutlak/Ekstrim lebih penting (Extreme Importance)	Bukti terkuat mendukung satu elemen atas lainnya

2.1.3 Rumus-Rumus AHP

$$\lambda_{maks} = \text{rata-rata dari } (A \cdot w) / w$$

$$CI = (\lambda_{maks} - n) / (n - 1)$$

$$CR = CI / RI$$

Keterangan: λ_{maks} = nilai eigen maksimum; n = jumlah kriteria; CI = *Consistency Index*; RI = *Random Index* (nilai dari tabel Saaty); CR = *Consistency Ratio*. Penilaian dinyatakan konsisten apabila $CR \leq 0,10$.

2.2 Skenario Kasus & Dataset

Skenario yang diangkat dalam proyek akhir ini adalah proses pemilihan smartwatch terbaik bagi konsumen yang dihadapkan pada ratusan pilihan produk di pasaran. Pengambil keputusan dalam sistem ini adalah konsumen umum atau individu yang ingin membeli smartwatch dengan pertimbangan yang rasional dan terukur, bukan hanya mengandalkan popularitas merek atau rekomendasi iklan semata.

Dataset yang digunakan dalam sistem ini bersumber dari file **Smart_watch_prices.csv** yang berisi data produk smartwatch dari berbagai merek global dengan total **379 baris data** dan 13 kolom atribut. Dataset ini mencakup merek-merek seperti Garmin, Fitbit, Mobvoi, Fossil, Amazfit, Samsung, Huawei, Xiaomi, Apple, Polar, dan lain-lain. Berikut adalah sampel 10 data pertama dari dataset tersebut:

Tabel 2.2 Sampel Dataset Smart_watch_prices.csv (10 Data Pertama)

Brand	Model	Harga	Baterai (hr)	Air (m)	HRM	GPS	OS
Apple	Watch Series 7	\$399	18	50	Ya	Ya	watchOS
Samsung	Galaxy Watch 4	\$249	40	50	Ya	Ya	Wear OS
Garmin	Venu 2	\$399	11	50	Ya	Ya	Garmin OS
Fitbit	Versa 3	\$229	6	50	Ya	Ya	Fitbit OS
Fossil	Gen 6	\$299	24	30	Ya	Ya	Wear OS
Huawei	Watch 3 Pro	\$499	14	50	Ya	Ya	HarmonyOS
TicWatch	E3	\$199	2	50	Ya	Ya	Wear OS
Amazfit	GTR 3	\$199	12	50	Ya	Ya	Amazfit OS
Skagen	Jorn Hybrid HR	\$195	14	30	Ya	Tidak	—
Mobvoi	TicWatch Pro 3 GPS	\$299	45	50	Ya	Ya	Wear OS

Sebelum digunakan dalam perhitungan AHP, dataset melalui proses preprocessing sebagai berikut:

- Kolom 'Heart Rate Monitor' dan 'GPS' dikonversi dari nilai teks ("Yes"/"No") menjadi nilai numerik (1/0) menggunakan fungsi preprocess() pada kode program.
- Kolom 'Price (USD)' dibersihkan dari karakter '\$' dan ',' menggunakan str.replace(), lalu dikonversi ke tipe numerik.
- Kolom 'Battery Life (days)' dan 'Water Resistance (meters)' dikonversi ke tipe numerik dengan pd.to_numeric(), nilai NaN digantikan dengan 0.
- Baris dengan nilai kosong (NaN) pada kolom kriteria utama dihapus menggunakan dropna() untuk menjamin integritas data.

2.3 Pemodelan Sistem Pendukung keputusan

2.3.1. Kriteria Penilaian

Sistem ini menggunakan lima kriteria dalam mengevaluasi setiap alternatif smartwatch. Jenis dan keterangan masing-masing kriteria disajikan pada Tabel 2.3.

Tabel 2.3 Daftar Kriteria

No	Kode	Nama Kriteria	Jenis	Keterangan
1	C1	Harga	Cost	Harga produk dalam USD; semakin murah semakin baik (diinvers sebelum normalisasi)
2	C2	Ketahanan Baterai	Benefit	Daya tahan baterai dalam hari; semakin lama semakin baik
3	C3	Ketahanan Air	Benefit	Kedalaman ketahanan air dalam meter; semakin tinggi semakin baik
4	C4	Heart Rate Monitor	Benefit	Ketersediaan sensor detak jantung (1=Ya, 0=Tidak)
5	C5	GPS	Benefit	Ketersediaan fitur GPS terintegrasi (1=Ya, 0=Tidak)

2.3.2. Alternatif

Alternatif dalam sistem ini adalah seluruh produk smartwatch yang tersedia dalam dataset setelah melalui proses preprocessing dan filter yang diterapkan pengguna. Dari total

379 data awal, jumlah alternatif aktif dapat bervariasi tergantung pada filter harga, brand, baterai minimum, dan ketersediaan fitur HRM/GPS yang dipilih pengguna melalui panel kontrol sidebar aplikasi.

2.3.3. Skala Penilaian Perbandingan Pasangan

Struktur hierarki AHP dalam sistem ini terdiri dari tiga tingkat: (1) Tujuan, yaitu memilih smartwatch terbaik; (2) Kriteria, yang meliputi lima kriteria C1 hingga C5; dan (3) Alternatif, yaitu seluruh produk smartwatch dalam dataset yang aktif. Pengguna mengonfigurasi bobot antarkriteria melalui mekanisme perbandingan berpasangan menggunakan slider interaktif yang disediakan di tab 'Pembobotan AHP'.

2.4 Perhitungan & Algoritma

2.4.1. Matriks Perbandingan Berpasangan

Sistem membangun matriks perbandingan berpasangan berukuran 5×5 (karena ada 5 kriteria) secara dinamis berdasarkan input slider dari pengguna. Nilai diagonal selalu bernilai 1 (setiap kriteria sama penting dengan dirinya sendiri), dan matriks bersifat resiprokal: jika $A[i][j] = k$, maka $A[j][i] = 1/k$. Berikut adalah ilustrasi struktur matriks perbandingan berpasangan:

Tabel 2.4 Ilustrasi Struktur Matriks Perbandingan Berpasangan (n=5)

Kriteria	Harga	Baterai	Ket. Air	HRM	GPS
Harga	1.0000	a_{ij}	a_{ij}	a_{ij}	a_{ij}
Baterai	$1/a_{ij}$	1.0000	a_{ij}	a_{ij}	a_{ij}
Ket. Air	$1/a_{ij}$	$1/a_{ij}$	1.0000	a_{ij}	a_{ij}
HRM	$1/a_{ij}$	$1/a_{ij}$	$1/a_{ij}$	1.0000	a_{ij}
GPS	$1/a_{ij}$	$1/a_{ij}$	$1/a_{ij}$	$1/a_{ij}$	1.0000

Implementasi pembangunan matriks pada kode program (SmartwatchAHP.py):

```
# Inisialisasi matriks identitas n×n
n = len(kriteria_list) # n = 5 kriteria
```

```

M = np.ones((n, n))      # diagonal = 1.0

# Mengisi matriks berdasarkan input slider pengguna
for (i, j, c1, c2) in pasangan:
    val = st.slider(label, min_value=-8, max_value=8,
value=0)
    if val < 0:
        sk = abs(val) + 1      # skala Saaty: 1-9
        M[i][j] = sk          # kriteria i lebih penting
        M[j][i] = 1/sk        # kebalikannya (resiprokal)
    elif val > 0:
        sk = val + 1
        M[i][j] = 1/sk        # kriteria j lebih penting
        M[j][i] = sk

# Inisialisasi matriks identitas n×n
n = len(kriteria_list)      # n = 5 kriteria
M = np.ones((n, n))      # diagonal = 1.0

# Mengisi matriks berdasarkan input slider pengguna
for (i, j, c1, c2) in pasangan:
    val = st.slider(label, min_value=-8, max_value=8,
value=0)
    if val < 0:
        sk = abs(val) + 1      # skala Saaty: 1-9
        M[i][j] = sk          # kriteria i lebih penting
        M[j][i] = 1/sk        # kebalikannya (resiprokal)
    elif val > 0:
        sk = val + 1
        M[i][j] = 1/sk        # kriteria j lebih penting

```

2.4.2. Normalisasi Matriks dan Perhitungan Bobot

Normalisasi dilakukan dengan membagi setiap elemen matriks dengan jumlah total kolomnya masing-masing, lalu bobot prioritas dihitung sebagai rata-rata setiap

baris matriks ternormalisasi. Implementasinya pada kode program adalah sebagai berikut:

```
# Normalisasi: bagi setiap elemen dengan jumlah kolomnya
kolom_sum = M.sum(axis=0)          # jumlah tiap kolom
M_norm    = M / kolom_sum          # matriks ternormalisasi

# Bobot prioritas = rata-rata tiap baris
bobot_akhir = M_norm.mean(axis=1)  # array bobot
[w1,w2,w3,w4,w5]
```

2.4.3. Perhitungan Consistency Ratio (CR)

Konsistensi penilaian divalidasi melalui perhitungan Consistency Index (CI) dan Consistency Ratio (CR). Nilai $CR \leq 0,10$ menunjukkan bahwa preferensi penilaian yang diinput pengguna bersifat logis dan konsisten. Tabel Random Index (RI) untuk $n=5$ adalah 1,12. Berikut implementasinya:

```
# Hitung lambda maksimum
vt      = M @ bobot_akhir          # vektor tertimbang
rasio_lam = vt / bobot_akhir       # rasio Aw/w
lambda_max = rasio_lam.mean()      # rata-rata rasio

# Consistency Index dan Ratio
ci = (lambda_max - n) / (n - 1)
ri_tbl = {1:0.00, 2:0.00, 3:0.58, 4:0.90, 5:1.12,
          6:1.24, 7:1.32, 8:1.41, 9:1.45, 10:1.49}
ri = ri_tbl.get(n, 1.12)           # n=5 → RI=1.12
cr = ci / ri                       # CR harus ≤ 0.10
```

2.4.4. Algoritma Perangkingan Alternatif

Setelah bobot kriteria diperoleh dan tervalidasi konsistensinya, sistem menghitung skor AHP untuk setiap alternatif produk smartwatch. Proses

normalisasi nilai kriteria menggunakan pendekatan yang berbeda untuk kriteria cost (harga) dan benefit (baterai, ketahanan air, HRM, GPS):

```
# Normalisasi nilai kriteria untuk setiap alternatif

# Harga (cost): diinvers agar produk lebih murah dapat skor lebih tinggi
inv_h = 1.0 / dn['Price (USD)'].replace(0, 0.01)
dn['N_Harga'] = inv_h / inv_h.sum()

# Benefit: nilai lebih besar = lebih baik
def benefit(col):
    s = dn[col].sum()
    return dn[col] / s if s != 0 else pd.Series(1/len(dn),
index=dn.index)

dn['N_Baterai'] = benefit('Battery Life (days)')
dn['N_Air'] = benefit('Water Resistance (meters)')
dn['N_HRM'] = benefit('Heart Rate Monitor')
dn['N_GPS'] = benefit('GPS')

# Hitung skor AHP: jumlah (nilai normalisasi × bobot)
df_res['Skor_AHP'] = (
    dn['N_Harga'] * bobot_akhir[0] + # C1 × w1
    dn['N_Baterai'] * bobot_akhir[1] + # C2 × w2
    dn['N_Air'] * bobot_akhir[2] + # C3 × w3
    dn['N_HRM'] * bobot_akhir[3] + # C4 × w4
    dn['N_GPS'] * bobot_akhir[4] + # C5 × w5
) * 100
```

Alternatif diurutkan berdasarkan nilai Skor_AHP dari yang tertinggi ke terendah, dan produk dengan skor tertinggi merupakan rekomendasi utama sistem. Hasil akhir perankingan ditampilkan dalam bentuk tabel interaktif dan podium visual di tab 'Hasil Rekomendasi'.

2.5 Implementasi & Tampilan Sistem

```
import streamlit as st
```

```

import pandas as pd
import numpy as np

st.set_page_config(
    page_title="SPK Smartwatch | AHP",
    layout="wide",
    initial_sidebar_state="expanded"
)

st.markdown("""
<style>
@import
url('https://fonts.googleapis.com/css2?family=Lora:wght@500;600;700&family=DM+Sans:ital,wght@0,300;0,400;0,500;0,600;1,300&family=DM+Mono:wght@400;500&display=swap');

/* ===== VARIABLES ===== */
:root {
    --bg:          #1C1A17;
    --surface:     #252220;
    --surface-2:   #2E2B27;
    --surface-3:   #363129;
    --border:      #3A3630;
    --border-light: #4A4540;

    --text-1: #EDE8E0;
    --text-2: #B8B0A4;
    --text-3: #7A7268;

    --sage:        #4E7A3A;
    --sage-mid:    #6A9E52;
    --sage-light:  #A8C990;
    --sage-pale:   #2A3D22;
    --sage-border: #3D5E2E;

    --amber:       #B8860B;
    --amber-light: #D4A832;
    --amber-pale:  #332500;
    --amber-border: #5C4000;

```

```

--rust:          #8B3A1E;
--rust-light:    #C45A30;
--rust-pale:     #2E1510;
--rust-border:   #5C2810;

--sidebar-bg:    #141210;
}

/* ===== GLOBAL ===== */
html, body, [class*="css"] {
    font-family: 'DM Sans', sans-serif;
    background-color: var(--bg) !important;
    color: var(--text-1);
}

.stApp { background-color: var(--bg) !important; }
.main { background-color: var(--bg) !important; }
.block-container {
    padding-top: 1.4rem !important;
    padding-bottom: 2.5rem !important;
    max-width: 1380px;
    background-color: var(--bg) !important;
}

/* ===== SIDEBAR ===== */
section[data-testid="stSidebar"] {
    background: var(--sidebar-bg) !important;
    border-right: 1px solid var(--border) !important;
}
section[data-testid="stSidebar"] * {
    color: var(--text-2) !important;
    font-family: 'DM Sans', sans-serif !important;
}
section[data-testid="stSidebar"] h1,
section[data-testid="stSidebar"] h2,
section[data-testid="stSidebar"] h3 {
    color: var(--text-1) !important;
    font-family: 'Lora', serif !important;
}

```

```

section[data-testid="stSidebar"] label {
    color: var(--text-3) !important;
    font-size: 0.75rem !important;
    font-weight: 500 !important;
    letter-spacing: 0.08em !important;
    text-transform: uppercase !important;
}
section[data-testid="stSidebar"] [data-testid="stMetricValue"] {
    color: var(--sage-light) !important;
    font-family: 'DM Mono', monospace !important;
    font-size: 1.35rem !important;
}
section[data-testid="stSidebar"] [data-testid="stMetricLabel"] {
    color: var(--text-3) !important;
    font-size: 0.7rem !important;
    text-transform: uppercase;
    letter-spacing: 0.06em;
}
section[data-testid="stSidebar"] hr {
    border-color: var(--border) !important;
    margin: 0.8rem 0;
}
section[data-testid="stSidebar"] .stCheckbox label {
    text-transform: none !important;
    letter-spacing: 0 !important;
    font-size: 0.85rem !important;
    color: var(--text-2) !important;
}
section[data-testid="stSidebar"] [data-
testid="stMetricContainer"] {
    background: var(--surface) !important;
    border: 1px solid var(--border) !important;
    border-top: 2px solid var(--sage) !important;
    border-radius: 3px !important;
    padding: 0.7rem 0.9rem !important;
}

/* ===== HEADER ===== */
.app-header {

```

```

        background: var(--sidebar-bg);
        border: 1px solid var(--border);
        border-bottom: 3px solid var(--sage);
        border-radius: 3px;
        padding: 1.8rem 2.2rem;
        margin-bottom: 1.4rem;
    }
    .app-header-label {
        font-size: 0.65rem;
        font-weight: 600;
        letter-spacing: 0.2em;
        text-transform: uppercase;
        color: var(--sage-light);
        margin-bottom: 0.5rem;
    }
    .app-header h1 {
        color: var(--text-1) !important;
        margin: 0 0 0.4rem;
        font-family: 'Lora', serif !important;
        font-size: 1.75rem;
        font-weight: 700;
        line-height: 1.2;
    }
    .app-header p {
        color: var(--text-3);
        margin: 0;
        font-size: 0.86rem;
        font-weight: 300;
        letter-spacing: 0.01em;
    }
}

/* ===== METRIC CARDS ===== */
[data-testid="metric-container"] {
    background: var(--surface) !important;
    border: 1px solid var(--border) !important;
    border-top: 3px solid var(--sage) !important;
    border-radius: 3px !important;
    padding: 0.9rem 1.1rem !important;
}

```

```

[data-testid="stMetricValue"] {
    color: var(--text-1) !important;
    font-family: 'DM Mono', monospace !important;
    font-weight: 500 !important;
    font-size: 1.4rem !important;
}
[data-testid="stMetricLabel"] {
    color: var(--text-3) !important;
    font-size: 0.69rem !important;
    text-transform: uppercase !important;
    letter-spacing: 0.08em !important;
    font-weight: 500 !important;
}

/* ===== TABS ===== */
.stTabs [data-baseweb="tab-list"] {
    gap: 0;
    background: transparent;
    border-bottom: 1px solid var(--border);
    border-radius: 0;
    padding: 0;
    margin-bottom: 1.4rem;
}
.stTabs [data-baseweb="tab"] {
    border-radius: 0;
    font-weight: 500;
    font-size: 0.78rem;
    letter-spacing: 0.08em;
    text-transform: uppercase;
    padding: 0.65rem 1.3rem;
    color: var(--text-3) !important;
    background: transparent !important;
    border-bottom: 2px solid transparent;
    margin-bottom: -1px;
}
.stTabs [aria-selected="true"] {
    background: transparent !important;
    color: var(--sage-light) !important;
    border-bottom: 2px solid var(--sage-mid) !important;
}

```

```

        font-weight: 600 !important;
    }

/* ===== STEP / FORMULA CARDS ===== */
.step-card {
    background: var(--surface-2);
    border: 1px solid var(--border);
    border-left: 3px solid var(--sage);
    border-radius: 2px;
    padding: 0.85rem 1.1rem;
    margin: 0.6rem 0;
    font-size: 0.84rem;
    line-height: 1.75;
    color: var(--text-2);
}

.formula-card {
    background: var(--amber-pale);
    border: 1px solid var(--amber-border);
    border-radius: 2px;
    padding: 0.9rem 1.2rem;
    font-family: 'DM Mono', monospace;
    font-size: 0.81rem;
    line-height: 1.85;
    margin: 0.6rem 0;
    color: var(--text-2);
}

.formula-card b { color: var(--amber-light); }

/* ===== CR BANNERS ===== */
.cr-banner {
    border-radius: 3px;
    padding: 1.2rem 1.5rem;
    margin: 0.8rem 0;
}

.cr-ok {
    background: var(--sage-pale);
    border: 1px solid var(--sage-border);
    border-left: 4px solid var(--sage-mid);
}

```

```

.cr-fail {
  background: var(--rust-pale);
  border: 1px solid var(--rust-border);
  border-left: 4px solid var(--rust-light);
}

.cr-status-label {
  font-size: 0.62rem;
  font-weight: 600;
  letter-spacing: 0.16em;
  text-transform: uppercase;
  margin-bottom: 0.25rem;
}

.cr-ok .cr-status-label { color: var(--sage-light); }
.cr-fail .cr-status-label { color: var(--rust-light); }

.cr-banner h4 { margin: 0 0 0.35rem; font-size: 0.95rem; font-weight: 600; }
.cr-ok h4 { color: var(--sage-light); }
.cr-fail h4 { color: var(--rust-light); }

.cr-banner p { margin: 0; font-size: 0.84rem; color: var(--text-2); line-height: 1.6; }

/* ===== WINNER CARDS ===== */

.winner-grid { display: grid; grid-template-columns: repeat(3,1fr); gap: 1rem; margin: 0.8rem 0; }
.winner-card {
  border-radius: 3px;
  padding: 1.3rem 1.4rem;
  text-align: center;
  position: relative;
}

.winner-gold { background: var(--amber-pale); border: 1px solid var(--amber-border); border-top: 3px solid var(--amber-light); }
.winner-silver { background: var(--surface-2); border: 1px solid var(--border-light); border-top: 3px solid var(--text-3); }
.winner-bronze { background: var(--rust-pale); border: 1px solid var(--rust-border); border-top: 3px solid var(--rust-light); }

.winner-place-label {
  font-size: 0.6rem;
  font-weight: 600;
}

```

```

    letter-spacing: 0.16em;
    text-transform: uppercase;
    margin-bottom: 0.5rem;
}
.winner-gold .winner-place-label { color: var(--amber-light); }
.winner-silver .winner-place-label { color: var(--text-3); }
.winner-bronze .winner-place-label { color: var(--rust-light); }
.winner-card .name {
    font-family: 'Lora', serif;
    font-weight: 600;
    font-size: 0.98rem;
    color: var(--text-1);
    margin: 0.2rem 0;
    line-height: 1.3;
}
.winner-card .score {
    font-family: 'DM Mono', monospace;
    font-size: 0.78rem;
    color: var(--text-3);
    margin-top: 0.45rem;
}

/* ===== SIDEBAR INFO BOX ===== */
.sidebar-info {
    background: rgba(78,122,58,0.06);
    border: 1px solid var(--sage-border);
    border-left: 2px solid var(--sage);
    border-radius: 2px;
    padding: 0.7rem 0.9rem;
    margin: 0.5rem 0;
    font-size: 0.79rem;
    color: var(--text-3);
    line-height: 1.65;
}
.sidebar-info b { color: var(--text-2); }

/* ===== DIVIDER ===== */
.divider {
    height: 1px;

```

```

        background: var(--border);
        margin: 1.4rem 0;
    }

/* ===== DATAFRAME ===== */
.stDataFrame {
    border-radius: 2px !important;
    border: 1px solid var(--border) !important;
    overflow: hidden;
}

/* Dark table headers */
[data-testid="stDataFrame"] th {
    background: var(--surface-3) !important;
    color: var(--text-3) !important;
    font-size: 0.72rem !important;
    letter-spacing: 0.07em !important;
    text-transform: uppercase !important;
    border-bottom: 1px solid var(--border) !important;
}

[data-testid="stDataFrame"] td {
    background: var(--surface) !important;
    color: var(--text-2) !important;
    border-bottom: 1px solid var(--border) !important;
    font-size: 0.85rem !important;
}

[data-testid="stDataFrame"] tr:hover td {
    background: var(--surface-2) !important;
}

/* ===== BUTTONS ===== */
.stButton > button {
    background: var(--sage) !important;
    color: var(--text-1) !important;
    border: none !important;
    border-radius: 2px !important;
    font-family: 'DM Sans', sans-serif !important;
    font-size: 0.8rem !important;
    font-weight: 600 !important;
    letter-spacing: 0.06em !important;
}

```

```

    text-transform: uppercase !important;
    padding: 0.5rem 1.2rem !important;
    transition: background 0.15s;
}
.stButton > button:hover {
    background: var(--sage-mid) !important;
}

/* ===== FORM / INPUT ===== */
.stTextInput input,
.stNumberInput input {
    background: var(--surface-2) !important;
    border: 1px solid var(--border-light) !important;
    border-radius: 2px !important;
    color: var(--text-1) !important;
    font-family: 'DM Sans', sans-serif !important;
    font-size: 0.88rem !important;
}
.stSelectbox > div > div {
    background: var(--surface-2) !important;
    border: 1px solid var(--border-light) !important;
    border-radius: 2px !important;
    color: var(--text-1) !important;
}
.stMultiSelect > div > div {
    background: var(--surface-2) !important;
    border: 1px solid var(--border-light) !important;
    border-radius: 2px !important;
}

/* ===== EXPANDER ===== */
.streamlit-expanderHeader {
    background: var(--surface-2) !important;
    border: 1px solid var(--border) !important;
    border-radius: 2px !important;
    color: var(--text-2) !important;
    font-size: 0.83rem !important;
    font-weight: 600 !important;
    letter-spacing: 0.04em !important;
}

```

```

}
.streamlit-expanderContent {
    background: var(--surface) !important;
    border: 1px solid var(--border) !important;
    border-top: none !important;
}

/* ===== ALERTS ===== */
.stWarning > div {
    background: var(--amber-pale) !important;
    border: 1px solid var(--amber-border) !important;
    border-left: 3px solid var(--amber-light) !important;
    color: var(--text-2) !important;
    border-radius: 2px !important;
}

.stInfo > div {
    background: var(--sage-pale) !important;
    border: 1px solid var(--sage-border) !important;
    border-left: 3px solid var(--sage-mid) !important;
    color: var(--text-2) !important;
    border-radius: 2px !important;
}

.stSuccess > div {
    background: var(--sage-pale) !important;
    border: 1px solid var(--sage-border) !important;
    border-left: 3px solid var(--sage-mid) !important;
    color: var(--text-2) !important;
    border-radius: 2px !important;
}

.stError > div {
    background: var(--rust-pale) !important;
    border: 1px solid var(--rust-border) !important;
    border-left: 3px solid var(--rust-light) !important;
    color: var(--text-2) !important;
    border-radius: 2px !important;
}

/* ===== RADIO ===== */
.stRadio > div { gap: 0.5rem; }

```

```

.stRadio label { font-size: 0.85rem !important; color: var(--text-
2) !important; }

/* ===== SLIDER ===== */
[data-baseweb="slider"] [role="slider"] { background: var(--sage-
mid) !important; }

/* ===== TYPOGRAPHY ===== */
h1, h2, h3, h4 {
    font-family: 'Lora', serif !important;
    color: var(--text-1) !important;
    letter-spacing: -0.01em;
}
h3 { font-size: 1.05rem !important; margin-bottom: 1rem; }
h4 { font-size: 0.92rem !important; margin-bottom: 0.7rem; }
p, span, li { font-family: 'DM Sans', sans-serif; color: var(--
text-2); }
.stCaption, small { color: var(--text-3) !important; font-size:
0.76rem !important; }
</style>
"""', unsafe_allow_html=True)

# =====
# SESSION STATE & DATA
# =====

@st.cache_data
def load_default_data():
    try:
        df = pd.read_csv("Smart_watch_prices.csv")
    except FileNotFoundError:
        data = {
            'Brand':
['Apple', 'Samsung', 'Garmin', 'Fitbit', 'Huawei', 'Apple', 'Samsung', '
Fossil', 'Xiaomi', 'Polar'],
            'Model':
['Watch Series 9', 'Galaxy Watch
6', 'Forerunner 255', 'Versa 4', 'GT 3 Pro',
            'Watch SE 2', 'Galaxy Watch 5', 'Gen 6', 'Mi
Band 8', 'Vantage V3'],

```

```

        'Price (USD)':
[399,299,349,229,249,249,279,295,49,599],
        'Battery Life (days)': [1.5,3,14,6,14,2,4,1,14,40],
        'Water Resistance (meters)':
[50,50,50,50,100,50,50,30,50,100],
        'Heart Rate Monitor':
['Yes','Yes','Yes','Yes','Yes','Yes','Yes','Yes','Yes','Yes'],
        'GPS':
['Yes','Yes','Yes','Yes','Yes','No','Yes','No','No','Yes'],
        'Display Size (mm)':
[45,44,46,40,46,40,44,44,np.nan,47],
        'OS Compatibility':
['iOS/Android','Android','iOS/Android','iOS/Android','Android',
'iOS/Android','Android','iOS/Android','Android','iOS/Android'],
    }
    return pd.DataFrame(data)
    return df

def preprocess(df):
    dc = df.copy()
    if 'Price (USD)' in dc.columns:
        dc['Price (USD)'] = pd.to_numeric(
            dc['Price (USD)'].astype(str)

        .str.replace('$','',regex=False).str.replace(',','',regex=False).
        str.strip(),
            errors='coerce')
    for col in ['Water Resistance (meters)','Battery Life
(days)']:
        if col in dc.columns:
            dc[col] = pd.to_numeric(dc[col],
errors='coerce').fillna(0)
    for col in ['Heart Rate Monitor','GPS']:
        if col in dc.columns:
            dc[col] =
dc[col].astype(str).str.strip().str.lower().apply(lambda x: 1 if
x=='yes' else 0)

```

```

    kriteria = ['Price (USD)', 'Battery Life (days)', 'Water
Resistance (meters)', 'Heart Rate Monitor', 'GPS']
    dc = dc.dropna(subset=[c for c in kriteria if c in dc.columns])
    return dc.reset_index(drop=True)

if 'df_master' not in st.session_state:
    st.session_state['df_master'] =
preprocess(load_default_data())
if 'crud_msg' not in st.session_state:
    st.session_state['crud_msg'] = None

df_master = st.session_state['df_master']

# =====
# SIDEBAR
# =====
with st.sidebar:
    st.markdown("### Panel Kontrol")
    st.markdown("---")

    st.markdown("#### Filter Produk")
    max_price = st.number_input("Maks. Harga (USD)",
min_value=10, value=1000, step=50)
    min_battery = st.number_input("Min. Baterai (Hari)",
min_value=0.0, value=0.0, step=1.0)
    brand_opts = sorted(df_master['Brand'].dropna().unique()) if
len(df_master) > 0 else []
    sel_brands = st.multiselect("Brand", options=brand_opts,
placeholder="Semua brand")
    has_hrm = st.checkbox("Wajib Heart Rate Monitor")
    has_gps = st.checkbox("Wajib GPS")

    df_filtered = df_master.copy()
    if sel_brands:
        df_filtered =
df_filtered[df_filtered['Brand'].isin(sel_brands)]
    df_filtered = df_filtered[df_filtered['Price (USD)'] <=
max_price]

```

```

df_filtered = df_filtered[df_filtered['Battery Life (days)']
>= min_battery]
if has_hrm:
    df_filtered = df_filtered[df_filtered['Heart Rate
Monitor'] == 1]
if has_gps:
    df_filtered = df_filtered[df_filtered['GPS'] == 1]

st.markdown("---")
st.markdown("#### Ringkasan")
c1, c2 = st.columns(2)
c1.metric("Produk", len(df_filtered))
c2.metric("Total DB", len(df_master))
if len(df_filtered) > 0:
    st.metric("Harga Rata-rata", f"${df_filtered['Price
(USD)'].mean():.0f}")
    pct_hrm = df_filtered['Heart Rate Monitor'].mean() * 100
    pct_gps = df_filtered['GPS'].mean() * 100
    st.markdown(f""<div class="sidebar-info">
HRM tersedia: <b>{pct_hrm:.0f}%</b> produk<br>
GPS tersedia: <b>{pct_gps:.0f}%</b> produk
</div>""", unsafe_allow_html=True)

st.markdown("---")
st.markdown("#### Tentang AHP")
st.markdown(f""<div class="sidebar-info">
<b>Analytical Hierarchy Process</b> (Saaty, 1980) adalah
metode pengambilan keputusan multi-kriteria
melalui perbandingan berpasangan antar kriteria.<br><br>
<b>Syarat konsistensi:</b> CR tidak boleh melebihi 0.10
</div>""", unsafe_allow_html=True)

# =====
# HEADER
# =====
st.markdown(f""
<div class="app-header">
<div class="app-header-label">Sistem Pendukung Keputusan</div>
<h1>Pemilihan Smartwatch</h1>

```

```

<p>Metode Analytical Hierarchy Process (AHP) &mdash; Pembobotan
kriteria dan perankingan alternatif produk smartwatch</p>
</div>
""" , unsafe_allow_html=True)

# =====
# TABS
# =====
tab1, tab_crud, tab_ahp, tab_hasil = st.tabs([
    "Eksplorasi Data",
    "Manajemen Data",
    "Pembobotan AHP",
    "Hasil Rekomendasi",
])

# =====
# TAB 1 - EKSPLORASI DATA
# =====
with tab1:
    if len(df_filtered) == 0:
        st.info("Tidak ada produk yang sesuai filter. Ubah
pengaturan di sidebar.")
    else:
        c1, c2, c3, c4, c5 = st.columns(5)
        c1.metric("Total Produk", f"{len(df_filtered)}")
        c2.metric("Harga Terendah", f"${df_filtered['Price
(USD)'].min():.0f}")
        c3.metric("Harga Tertinggi", f"${df_filtered['Price
(USD)'].max():.0f}")
        c4.metric("Baterai Terlama", f"{df_filtered['Battery Life
(days)'].max():.0f} hr")
        c5.metric("Rata-rata Harga", f"${df_filtered['Price
(USD)'].mean():.0f}")

        st.markdown('<div class="divider"></div>',
unsafe_allow_html=True)

        col_tbl, col_charts = st.columns([3, 2], gap="large")
        with col_tbl:

```

```

st.markdown("#### Daftar Produk")
dd = df_filtered.copy()
dd['Heart Rate Monitor'] = dd['Heart Rate
Monitor'].map({1:'Ya', 0:'Tidak'})
dd['GPS'] = dd['GPS'].map({1:'Ya',
0:'Tidak'})
show = [c for c in ['Brand', 'Model', 'Price
(USD)', 'Battery Life (days)',
'Water Resistance
(meters)', 'Heart Rate Monitor', 'GPS'] if c in dd.columns]
st.dataframe(dd[show].reset_index(drop=True),
use_container_width=True, hide_index=True,
column_config={
    "Price (USD)":
st.column_config.NumberColumn("Harga (USD)", format="$%.2f"),
    "Battery Life (days)":
st.column_config.NumberColumn("Baterai (Hari)", format="%.1f"),
    "Water Resistance (meters)":
st.column_config.NumberColumn("Tahan Air (m)", format="%d m"),
})

with col_charts:
    st.markdown("#### Distribusi Harga per Brand")
    st.bar_chart(df_filtered.groupby('Brand')['Price
(USD)'].mean().sort_values(ascending=False),
use_container_width=True,
color="#4E7A3A")
    st.markdown("#### Rata-rata Baterai per Brand")
    st.bar_chart(df_filtered.groupby('Brand')['Battery
Life (days)'].mean().sort_values(ascending=False),
use_container_width=True,
color="#B8860B")

st.markdown('<div class="divider"></div>',
unsafe_allow_html=True)
st.markdown("#### Statistik Deskriptif")
stat_cols = [c for c in ['Price (USD)', 'Battery Life
(days)', 'Water Resistance (meters)']
if c in df_filtered.columns]

```

```

        st.dataframe(df_filtered[stat_cols].describe().round(2),
use_container_width=True)

# =====
# TAB CRUD
# =====
with tab_crud:
    if st.session_state['crud_msg']:
        mtype, mtxt = st.session_state['crud_msg']
        if mtype == 'success': st.success(mtxt)
        else: st.error(mtxt)
        st.session_state['crud_msg'] = None

    st.markdown("#### Pilih Operasi")
    aksi = st.radio("", ["Tambah", "Lihat Semua", "Edit",
"Hapus"],
                    horizontal=True,
label_visibility="collapsed")
    st.markdown('<div class="divider"></div>',
unsafe_allow_html=True)

    if aksi == "Tambah":
        st.markdown("#### Tambah Produk Baru")
        with st.form("frm_tambah", clear_on_submit=True):
            st.markdown("***Informasi Dasar**")
            r1, r2, r3 = st.columns(3)
            brand_n = r1.text_input("Brand", placeholder="cth:
Apple")
            model_n = r2.text_input("Model", placeholder="cth:
Watch Series 9")
            harga_n = r3.number_input("Harga (USD)",
min_value=1.0, value=299.0, step=10.0)
            st.markdown("***Spesifikasi Teknis**")
            s1, s2, s3, s4, s5 = st.columns(5)
            bat_n = s1.number_input("Baterai (Hari)",
min_value=0.1, value=3.0, step=0.5)
            air_n = s2.number_input("Tahan Air (m)",
min_value=0, value=50, step=10)

```

```

        hrm_n      = s3.selectbox("Heart Rate Monitor",
["Yes", "No"])

        gps_n  = s4.selectbox("GPS", ["Yes", "No"])
        disp_n = s5.number_input("Layar (mm)", min_value=0.0,
value=44.0, step=1.0)

        os_n      = st.selectbox("Kompatibilitas OS",
["iOS/Android", "Android", "iOS"])

        if st.form_submit_button("Simpan Produk",
use_container_width=True, type="primary"):
            if not brand_n.strip() or not model_n.strip():
                st.session_state['crud_msg'] = ('error',
'Brand dan Model tidak boleh kosong.')
                st.rerun()
            else:
                baru = {c: np.nan for c in df_master.columns}
                baru.update({'Brand': brand_n.strip(),
'Model': model_n.strip(),
'Price (USD)': float(harga_n),
'Battery Life (days)': float(bat_n),
'Water Resistance (meters)':
int(air_n),
'Heart Rate Monitor': 1 if
hrm_n=='Yes' else 0,
'GPS': 1 if gps_n=='Yes' else 0})
                if 'Display Size (mm)' in df_master.columns:
baru['Display Size (mm)'] = float(disp_n)
                if 'OS Compatibility' in df_master.columns:
baru['OS Compatibility'] = os_n
                st.session_state['df_master'] = pd.concat(
[df_master, pd.DataFrame([baru])],
ignore_index=True)
                st.session_state['crud_msg'] = ('success',
f"'{brand_n} {model_n}' berhasil ditambahkan.")
                st.rerun()

        elif aksi == "Lihat Semua":
            st.markdown("#### Semua Data Tersimpan")
            if len(df_master) == 0:
                st.info("Belum ada data.")

```

```

else:
    dv = df_master.copy().reset_index().rename(columns={"index": "ID"})
    dv['Heart Rate Monitor'] = dv['Heart Rate Monitor'].map({1: 'Ya', 0: 'Tidak'})
    dv['GPS'] = dv['GPS'].map({1: 'Ya', 0: 'Tidak'})
    st.dataframe(dv, use_container_width=True, hide_index=True,
        column_config={"Price (USD)": st.column_config.NumberColumn("Harga (USD)", format="$%.2f"),
            "Battery Life (days)": st.column_config.NumberColumn("Baterai (Hari)", format="%.1f")})
    st.caption(f"Total {len(df_master)} produk tersimpan.")

elif aksi == "Edit":
    st.markdown("#### Edit Data Produk")
    if len(df_master) == 0:
        st.info("Belum ada data.")
    else:
        dr = df_master.reset_index(drop=True)
        opts = [f"[{i}] {r['Brand']} {r['Model']} - ${r['Price (USD)']:.2f}" for i, r in dr.iterrows()]
        sel = st.selectbox("Pilih produk:", opts)
        idx = int(sel.split(" ")[0].replace("[", "").strip())
        row = dr.iloc[idx]
        st.markdown('<div class="divider"></div>', unsafe_allow_html=True)
        with st.form("frm_edit"):
            st.markdown("***Informasi Dasar**")
            e1, e2, e3 = st.columns(3)
            b_e = e1.text_input("Brand", value=str(row.get('Brand', '')))
            m_e = e2.text_input("Model", value=str(row.get('Model', '')))
            h_e = e3.number_input("Harga", min_value=1.0, value=float(row.get('Price (USD)', 0)), step=10.0)
            st.markdown("***Spesifikasi Teknis**")

```

```

        t1, t2, t3, t4 = st.columns(4)
        ba_e = t1.number_input("Baterai (Hari)",
min_value=0.1, value=float(row.get('Battery Life (days)',1)),
step=0.5)
        ai_e = t2.number_input("Tahan Air (m)",
min_value=0, value=int(row.get('Water Resistance (meters)',0)),
step=10)
        hr_e = t3.selectbox("HRM", ["Yes","No"], index=0
if row.get('Heart Rate Monitor',0)==1 else 1)
        gp_e = t4.selectbox("GPS", ["Yes","No"], index=0
if row.get('GPS',0)==1 else 1)
        if st.form_submit_button("Simpan Perubahan",
use_container_width=True, type="primary"):
            dm = st.session_state['df_master']
            for col, val in {'Brand': b_e.strip(),
'Price (USD)': float(h_e),
'Battery Life (days)': float(ba_e),
'Water Resistance (meters)':
int(ai_e),
'Heart Rate Monitor': 1 if
hr_e=='Yes' else 0,
'GPS': 1 if gp_e=='Yes' else
0}.items():
                if col in dm.columns:
                    dm.iloc[idx, dm.columns.get_loc(col)]
= val
            st.session_state['df_master'] = dm
            st.session_state['crud_msg'] = ('success',
f"'{b_e} {m_e}' berhasil diperbarui.")
            st.rerun()

    elif aksi == "Hapus":
        st.markdown("#### Hapus Produk")
        if len(df_master) == 0:
            st.info("Tidak ada data untuk dihapus.")
        else:
            dr = df_master.reset_index(drop=True)

```

```

        opts = [f"[{i}]      {r['Brand']} {r['Model']}" for i, r in dr.iterrows()]
    ${r['Price (USD)']:.2f}" for i, r in dr.iterrows()]
    sel = st.selectbox("Pilih produk:", opts)
    idx = int(sel.split(" ")[0].replace("[", "").strip())
    row = dr.iloc[idx]
    st.warning(f"Anda akan menghapus: **{row['Brand']} {row['Model']}** - ${row['Price (USD)']:.2f}")
    col_del, _ = st.columns([1, 4])
    with col_del:
        if st.button("Konfirmasi Hapus", type="primary",
use_container_width=True):
            nama = f"{row['Brand']} {row['Model']}"
            st.session_state['df_master'] =
dr.drop(index=idx).reset_index(drop=True)
            st.session_state['crud_msg'] = ('success',
f"'{nama}' berhasil dihapus.")
            st.rerun()

# =====
# TAB AHP
# =====

with tab_ahp:
    kriteria_list = ["Harga", "Baterai", "Ketahanan Air", "Detak
Jantung", "GPS"]
    n = len(kriteria_list)
    M = np.ones((n, n))

    pasangan = [
        (0,1,"Harga","Baterai"),          (0,2,"Harga","Ketahanan
Air"),
        (0,3,"Harga","Detak Jantung"),    (0,4,"Harga","GPS"),
        (1,2,"Baterai","Ketahanan Air"),   (1,3,"Baterai","Detak
Jantung"),
        (1,4,"Baterai","GPS"),              (2,3,"Ketahanan Air","Detak
Jantung"),
        (2,4,"Ketahanan Air","GPS"),        (3,4,"Detak
Jantung","GPS"),
    ]

```

```

with st.expander("Panduan Skala Saaty", expanded=False):
    st.dataframe(pd.DataFrame({
        "Nilai": [1,2,3,4,5,6,7,8,9],
        "Definisi": ["Sama penting","Antara sama dan sedikit
lebih penting",
                    "Sedikit lebih penting","Antara sedikit
dan cukup penting",
                    "Cukup lebih penting","Antara cukup dan
sangat penting",
                    "Sangat lebih penting","Antara sangat dan
mutlak penting",
                    "Mutlak/Ekstrim lebih penting"],
    })), use_container_width=True, hide_index=True)

st.markdown("#### Perbandingan Berpasangan Antar Kriteria")
st.caption("Geser ke kiri jika kriteria pertama lebih penting,
ke kanan jika kriteria kedua lebih penting.")

pw_cols = st.columns(2)
for idx_p, (i, j, c1, c2) in enumerate(pasangan):
    with pw_cols[idx_p % 2]:
        lc, rc = st.columns([2, 1])
        lc.markdown(
            f"<span                style='font-size:0.85rem;font-
weight:600;color:var(--text-1)'>{c1}</span>"
            f"<span                style='color:var(--text-3);font-
size:0.8rem'> vs </span>"
            f"<span                style='font-size:0.85rem;font-
weight:600;color:var(--text-1)'>{c2}</span>",
            unsafe_allow_html=True)
        val = st.slider("", min_value=-8, max_value=8,
value=0,
                        key=f"s_{i}_{j}",
label_visibility="collapsed")
        if val < 0:
            sk = abs(val)+1; M[i][j] = sk; M[j][i] = 1/sk
            rc.markdown(f"<span                style='color:var(--sage-
light);font-size:0.76rem;font-weight:600'>{c1}                ×{sk}</span>",
unsafe_allow_html=True)

```

```

        elif val > 0:
            sk = val+1; M[i][j] = 1/sk; M[j][i] = sk
            rc.markdown(f"<span style='color:var(--sage-light);font-size:0.76rem;font-weight:600'>{c2} ×{sk}</span>",
unsafe_allow_html=True)
        else:
            rc.markdown("<span style='color:var(--text-3);font-size:0.76rem'>Setara</span>", unsafe_allow_html=True)

# AHP calc
kolom_sum = M.sum(axis=0)
M_norm = M / kolom_sum
bobot_akhir = M_norm.mean(axis=1)
vt = M @ bobot_akhir
rasio_lam = vt / bobot_akhir
lambda_max = rasio_lam.mean()
ci = (lambda_max - n) / (n - 1)
ri_tbl =
{1:0.00,2:0.00,3:0.58,4:0.90,5:1.12,6:1.24,7:1.32,8:1.41,9:1.45,10:1.49}
ri = ri_tbl.get(n, 1.12)
cr = ci / ri if ri != 0 else 0.0

st.markdown('<div class="divider"></div>',
unsafe_allow_html=True)
st.markdown("### Detail Perhitungan AHP")

with st.expander("Langkah 1 – Matriks Perbandingan Berpasangan (A)", expanded=True):
    st.markdown('<p>Matriks simetris resiprokal: diagonal = 1, elemen A[j,i] = 1/A[i,j].</p>', unsafe_allow_html=True)
    df_M = pd.DataFrame(M, index=kriteria_list, columns=kriteria_list)

st.dataframe(df_M.style.format("{:.4f}").background_gradient(cmap='YlGn', axis=None),
use_container_width=True)

```

```

        st.markdown('<div class="step-card">Sifat resiprokal:
jika A lebih penting dari B dengan skala 3, maka B terhadap A =
1/3 = 0.333</div>', unsafe_allow_html=True)

    with st.expander("Langkah 2 – Normalisasi dan Bobot
Prioritas", expanded=True):
        l2a, l2b = st.columns([3, 2], gap="large")
        with l2a:
            st.markdown("***Jumlah Tiap Kolom***")
            st.dataframe(pd.DataFrame([kolom_sum],
columns=kriteria_list, index=["Jumlah
Kolom"]).style.format("{:.4f}"),
                        use_container_width=True)
            st.markdown("***Matriks Ternormalisasi** *(elemen
dibagi jumlah kolom)***")
            st.dataframe(pd.DataFrame(M_norm,
index=kriteria_list,
columns=kriteria_list).style.format("{:.4f}").background_gradient
(cmap='YlGn', axis=None),
                        use_container_width=True)
        with l2b:
            st.markdown("***Bobot Prioritas** *(rata-rata
baris)***")
            df_w = pd.DataFrame({"Kriteria": kriteria_list,
                                "Bobot":
np.round(bobot_akhir, 6),
                                "Bobot (%)":
np.round(bobot_akhir*100, 3)}).sort_values("Bobot (%)",
ascending=False)
            st.dataframe(df_w, use_container_width=True,
hide_index=True,
                                column_config={"Bobot (%)":
st.column_config.ProgressColumn(
                                "Bobot (%)", min_value=0, max_value=100,
format="%.2f%%")})
            st.markdown('<div class="formula-card">Jumlah Bobot =
<b>{:.6f}</b> &asymp; 1.000</div>'.format(bobot_akhir.sum()),
unsafe_allow_html=True)

```

```

        st.bar_chart(pd.DataFrame({"Bobot": bobot_akhir*100, index=kriteria_list},
                                use_container_width=True,
                                color="#4E7A3A"))

        with st.expander("Langkah 3 – Perhitungan Lambda Maksimum",
expanded=True):
            st.dataframe(pd.DataFrame({
                "Kriteria": kriteria_list,
                "Vektor Tertimbang (Aw)": np.round(vt,6),
                "Bobot (w)": np.round(bobot_akhir,6),
                "Rasio (Aw/w)": np.round(rasio_lam,6),
            }), use_container_width=True, hide_index=True)
            st.markdown(f"""<div class="formula-card">
lambda_max = ({' ' + '.join([f"{r:.4f}" for r in
rasio_lam])}) / {n}<br>
lambda_max = {rasio_lam.sum():.6f} / {n}<br>
lambda_max = <b>{lambda_max:.6f}</b>
</div>""", unsafe_allow_html=True)
            st.markdown('<div class="step-card">Semakin dekat
lambda_max dengan n, semakin konsisten matriks. Jika lambda_max =
n, maka konsistensi sempurna.</div>', unsafe_allow_html=True)

        with st.expander("Langkah 4 – CI, RI, dan CR", expanded=True):
            st.markdown("**Tabel Random Index (RI) – Saaty, 1980**")
            ri_html = "<div style='display:flex;gap:5px;flex-
wrap:wrap;margin:0.5rem 0 1rem'>"
            for ni, riv in ri_tbl.items():
                active = ni == n
                bg = "var(--sage)" if active else "var(--
surface-3)"
                fc = "var(--text-1)" if active else "var(--text-
3)"
                brd = "var(--sage-mid)" if active else "var(--
border)"
                ri_html += (f"<div
style='background:{bg};color:{fc};border:1px solid {brd};"
f"border-radius:2px;padding:0.35rem
0.7rem;font-family:DM Mono,monospace;"

```

```

f"font-size:0.78rem;font-weight:600;text-align:center;min-width:52px'"
f"n={ni}<br>{riv}</div>")
ri_html += "</div>"
st.markdown(ri_html, unsafe_allow_html=True)
st.caption(f"Kotak terang = nilai yang digunakan: n = {n},
RI = {ri}")

st.markdown('<div class="divider"></div>',
unsafe_allow_html=True)
col_ci, col_ri, col_cr = st.columns(3)
with col_ci:
    st.markdown("##### Consistency Index (CI)")
    st.markdown(f""<div class="formula-card">
CI = (lambda_max &minus; n) / (n &minus; 1)<br>
CI = ({lambda_max:.4f} &minus; {n}) / ({n} &minus;
1)<br>
CI = {lambda_max-n:.6f} / {n-1}<br>
CI = <b>{ci:.6f}</b>
</div>""", unsafe_allow_html=True)
    st.info("CI = 0 artinya matriks sempurna konsisten.")
with col_ri:
    st.markdown("##### Random Index (RI)")
    st.markdown(f""<div class="formula-card">
n = {n} kriteria<br>
Lookup tabel Saaty:<br>
RI = <b>{ri}</b>
</div>""", unsafe_allow_html=True)
    st.info("RI = rata-rata CI dari 500 matriks acak
berukuran n&times;n.")
with col_cr:
    st.markdown("##### Consistency Ratio (CR)")
    st.markdown(f""<div class="formula-card">
CR = CI / RI<br>
CR = {ci:.6f} / {ri}<br>
CR = <b>{cr:.6f}</b> ({cr*100:.2f}%)
</div>""", unsafe_allow_html=True)

```



```

with tab_hasil:
    if cr > 0.10:
        st.warning(f"CR = {cr:.4f} melebihi 0.10. Bobot tidak
konsisten – hasil mungkin kurang representatif. Perbaiki penilaian
di tab Pembobotan AHP.")

    if len(df_filtered) == 0:
        st.info("Tidak ada produk sesuai filter. Ubah pengaturan
di sidebar.")
    else:
        dn = df_filtered.copy().reset_index(drop=True)
        inv_h = 1.0 / dn['Price (USD)'].replace(0, 0.01)
        dn['N_Harga'] = inv_h / inv_h.sum()

        def benefit(col):
            s = dn[col].sum()
            return dn[col] / s if s != 0 else pd.Series(1/len(dn),
index=dn.index)

        dn['N_Baterai'] = benefit('Battery Life (days)')
        dn['N_Air']      = benefit('Water Resistance (meters)')
        dn['N_HRM']      = benefit('Heart Rate Monitor')
        dn['N_GPS']      = benefit('GPS')

        df_res = df_filtered.copy()
        df_res['Skor_AHP'] = (
            dn['N_Harga'] * bobot_akhir[0] +
            dn['N_Baterai'] * bobot_akhir[1] +
            dn['N_Air'] * bobot_akhir[2] +
            dn['N_HRM'] * bobot_akhir[3] +
            dn['N_GPS'] * bobot_akhir[4]
        ) * 100

        df_final = df_res.sort_values('Skor_AHP',
ascending=False).reset_index(drop=True)
        df_final.index += 1
        df_final =
df_final.reset_index().rename(columns={"index": "Peringkat"})

```

```

st.markdown("#### Podium Rekomendasi")
podium = st.columns(3)
medals = [
    ("Peringkat Pertama", "winner-gold"),
    ("Peringkat Kedua", "winner-silver"),
    ("Peringkat Ketiga", "winner-bronze"),
]
for i, (lbl, cls) in enumerate(medals):
    if len(df_final) > i:
        r = df_final.iloc[i]
        with podium[i]:
            st.markdown(f"""<div class="winner-card
{cls}">
                <div class="winner-place-label">{lbl}</div>
                <div
class="name">{r['Brand']}<br>{r['Model']}</div>
                <div class="score">${r['Price (USD)']:.0f}
&nbsp;&bull;&nbsp;&nbsp; Skor {r['Skor_AHP']:.3f}</div>
                </div>""", unsafe_allow_html=True)

            st.markdown('<div class="divider"></div>',
unsafe_allow_html=True)

col_tbl2, col_info = st.columns([3, 1], gap="large")
with col_tbl2:
    st.markdown(f"#### Top 10 dari {len(df_final)}
Produk")
    show_cols = [c for c in
['Peringkat', 'Brand', 'Model', 'Price (USD)',
'Battery Life (days)', 'Water
Resistance (meters)',
'Heart Rate
Monitor', 'GPS', 'Skor_AHP'] if c in df_final.columns]
    df_show = df_final[show_cols].head(10).copy()
    df_show['Heart Rate Monitor'] = df_show['Heart Rate
Monitor'].map({1:'Ya', 0:'Tidak'})
    df_show['GPS'] =
df_show['GPS'].map({1:'Ya', 0:'Tidak'})

```

```

        st.dataframe(df_show, use_container_width=True,
hide_index=True,
        column_config={
            "Price (USD)":
st.column_config.NumberColumn("Harga (USD)", format="$%.2f"),
            "Battery Life (days)":
st.column_config.NumberColumn("Baterai (hr)", format="%.1f"),
            "Water Resistance (meters)":
st.column_config.NumberColumn("Tahan Air (m)"),
            "Skor_AHP": st.column_config.ProgressColumn(
                "Skor AHP", min_value=0,
max_value=float(df_final['Skor_AHP'].max()), format="%.3f"),
        })
    with col_info:
        st.markdown("#### Bobot Digunakan")
        df_w_show = pd.DataFrame({"Kriteria": kriteria_list,
            "Bobot (%)":
np.round(bobot_akhir*100, 2)
        }).sort_values("Bobot (%)",
ascending=False)
        st.dataframe(df_w_show, use_container_width=True,
hide_index=True,
        column_config={"Bobot (%)":
st.column_config.ProgressColumn(
            "Bobot (%)", min_value=0, max_value=100,
format="%.1f%%")})
        status_cr = "Konsisten" if cr <= 0.1 else "Tidak
Konsisten"
        st.markdown(f'<div class="step-card">CR =
<b>{cr:.4f}</b><br>{status_cr}</div>', unsafe_allow_html=True)

        st.markdown('<div class="divider"></div>',
unsafe_allow_html=True)
        st.markdown("#### Visualisasi Top 10")
        vc1, vc2 = st.columns(2)
        with vc1:
            st.markdown("***Skor AHP**")
            top10 = df_final.head(10).copy()

```

```

top10['Nama'] = top10['Brand'] + " " + top10['Model']

st.bar_chart(top10.sort_values('Skor_AHP').set_index('Nama')['Skor_AHP'],
              use_container_width=True,
              color="#4E7A3A")
    with vc2:
        st.markdown("**Harga vs Skor AHP**")
        sc = df_final.head(10).copy()
        sc['Nama'] = sc['Brand'] + " " + sc['Model']
        st.scatter_chart(sc.set_index('Nama')[['Price
(USD)', 'Skor_AHP']],
                        x='Price      (USD)', y='Skor_AHP',
                        use_container_width=True)

        with st.expander("Detail      Skor      Normalisasi      Tiap
Alternatif", expanded=False):
            dn_d =
dn[['N_Harga', 'N_Baterai', 'N_Air', 'N_HRM', 'N_GPS']].copy()
            dn_d.columns =
['Norm.Harga', 'Norm.Baterai', 'Norm.Tahan
Air', 'Norm.HRM', 'Norm.GPS']
            dn_d.insert(0, 'Brand', df_filtered['Brand'].values)
            dn_d.insert(1, 'Model', df_filtered['Model'].values)
            dn_d['Skor_AHP'] = df_res['Skor_AHP'].values
            dn_d = dn_d.sort_values('Skor_AHP',
ascending=False).reset_index(drop=True)
            st.dataframe(dn_d.style.format({
                c: "{:.5f}" for c in dn_d.columns if c not in
['Brand', 'Model']
            }).background_gradient(cmap='YlGn',
subset=['Skor_AHP']),
            use_container_width=True, hide_index=True)
            st.markdown("""<div class="step-card">
<b>Metode normalisasi:</b><br>
        &nbsp;&nbsp;&nbsp;Harga      (Cost): diinvers      (1/harga) lalu
dibagi total &mdash; produk lebih murah mendapat skor lebih
tinggi.<br>

```

Benefit lainnya: dibagi total kolom
— nilai lebih besar mendapat skor lebih tinggi.

""", unsafe_allow_html=True)

Tangkapan Layar sistem

Panel Kontrol

Filter Produk

MAKS. HARGA (USD)

1000

MIN. BATERAI (HARI)

000

BRAND

Semua brand

Wajib Heart Rate Monitor

Wajib GPS

Ringkasan

PRODUK

372

TOTAL DB

378

HARGA RATA-RATA

\$295

HRM tersedia: 100% produk

GPS tersedia: 92% produk

Tentang AHP

SISTEM PENDUKUNG KEPUTUSAN

Pemilihan Smartwatch

Metode Analytical Hierarchy Process (AHP) – Pembobotan kriteria dan perankingan alternatif produk smartwatch

EKSPLORASI DATA

MANAJEMEN DATA

PEMBOBOTAN AHP

HASIL REKOMENDASI

TOTAL PRODUK

372

HARGA TERENDAH

\$49

HARGA TERTINGGI

\$990

BATERAI TERLAMBA

72 hr

RATA-RATA HARGA

\$295

Daftar Produk

Brand	Model	Harga (USD)	Baterai (Hari)	Tahan Air (m)	Heart Rate Monitor	GPS
Apple	Watch Series 7	\$399.00	18.0	50 m	Ya	Ya
Samsung	Galaxy Watch 4	\$249.00	40.0	50 m	Ya	Ya
Garmin	Venu 2	\$399.00	11.0	50 m	Ya	Ya
Fitbit	Versa 3	\$229.00	6.0	50 m	Ya	Ya
Fossil	Gen 6	\$299.00	24.0	30 m	Ya	Ya
Huawei	Watch 3 Pro	\$499.00	14.0	50 m	Ya	Ya
TicWatch	E3	\$199.00	2.0	50 m	Ya	Ya
Oppe	Watch 2	\$199.00	4.0	50 m	Ya	Ya

Distribusi Harga per Brand

Panel Kontrol

Filter Produk

MAKS. HARGA (USD)

1000

MIN. BATERAI (HARI)

000

BRAND

Semua brand

Wajib Heart Rate Monitor

Wajib GPS

Ringkasan

PRODUK

372

TOTAL DB

378

HARGA RATA-RATA

\$295

HRM tersedia: 100% produk

GPS tersedia: 92% produk

Tentang AHP

EKSPLORASI DATA

MANAJEMEN DATA

PEMBOBOTAN AHP

HASIL REKOMENDASI

Pilih Operasi

Tambah

Lihat Semua

Edit

Hapus

Tambah Produk Baru

Informasi Dasar

Brand

Model

Harga (USD)

cthr Apple

cthr Watch Series 9

299.00

Spesifikasi Teknis

Baterai (Hari)

Tahan Air (m)

Heart Rate Monitor

GPS

Layar (mm)

3.00

50

Yes

Yes

44.00

Kompatibilitas OS

iOS/Android

Simpan Produk

Panel Kontrol

Filter Produk

MAKS. HARGA (USD)

1000

- +

MIN. BATERAI (HARI)

000

- +

BRAND

Semua brand

▼

Wajib Heart Rate Monitor

Wajib GPS

Ringkasan

PRODUK

372

TOTAL DB

378

HARGA RATA-RATA

\$295

HRM tersedia: 100% produk

GPS tersedia: 92% produk

EKSPLORASI DATA

MANAJEMEN DATA

PEMBOBOTAN AHP

HASIL REKOMENDASI

Deploy

Pilih Operasi

Tambah

Lihat Semua

Edit

Hapus

Semua Data Tersimpan

ID	Brand	Model	Operating System	Connectivity	Display Type	Display Size (inches)	Resolution	Water Resistance (meters)	Baterai (Hari)	Heart Rate Monitor	GPS
0	Apple	Watch Series 7	watchOS	Bluetooth, Wi-Fi, Cellular	Retina	1.9	396 x 484	50	18.0	Ya	Ya
1	Samsung	Galaxy Watch 4	Wear OS	Bluetooth, Wi-Fi, Cellular	AMOLED	1.4	450 x 450	50	40.0	Ya	Ya
2	Garmin	Venu 2	Garmin OS	Bluetooth, Wi-Fi	AMOLED	1.3	416 x 416	50	11.0	Ya	Ya
3	Fitbit	Versa 3	Fitbit OS	Bluetooth, Wi-Fi	AMOLED	1.58	336 x 336	50	6.0	Ya	Ya
4	Fossil	Gen 6	Wear OS	Bluetooth, Wi-Fi	AMOLED	1.28	416 x 416	30	24.0	Ya	Ya
5	Huawei	Watch 3 Pro	HarmonyOS	Bluetooth, Wi-Fi, Cellular	AMOLED	1.43	466 x 466	50	14.0	Ya	Ya
6	TicWatch	E3	Wear OS	Bluetooth, Wi-Fi, Cellular	IPS LCD	1.3	360 x 360	50	2.0	Ya	Ya
7	Oppe	Watch 2	ColorOS	Bluetooth, Wi-Fi	AMOLED	1.75	372 x 430	50	4.0	Ya	Ya
8	Amazfit	GTR 3	Amazfit OS	Bluetooth, Wi-Fi	AMOLED	1.39	454 x 454	50	12.0	Ya	Ya
9	Skagen	Jorn Hybrid HR	None	Bluetooth	None	None	None	30	14.0	Ya	Ti

Total 378 produk tersimpan.

Panel Kontrol

Filter Produk

MAKS. HARGA (USD)

1000

- +

MIN. BATERAI (HARI)

000

- +

BRAND

Semua brand

▼

Wajib Heart Rate Monitor

Wajib GPS

Ringkasan

PRODUK

372

TOTAL DB

378

HARGA RATA-RATA

\$295

HRM tersedia: 100% produk

GPS tersedia: 92% produk

EKSPLORASI DATA

MANAJEMEN DATA

PEMBOBOTAN AHP

HASIL REKOMENDASI

Deploy

Pilih Operasi

Tambah

Lihat Semua

Edit

Hapus

Edit Data Produk

Pilih produk:

[0] Apple Watch Series 7 — \$399.00

▼

Informasi Dasar

Brand

Model

Harga

Apple

Watch Series 7

399.00

- +

Spesifikasi Teknis

Baterai (Hari)

Tahan Air (m)

HRM

GPS

18.00

- +

50

- +

Yes

▼

Yes

▼

Simpan Perubahan

Panel Kontrol

Filter Produk

MAKS. HARGA (USD)

1000

- +

MIN. BATERAI (HARI)

000

- +

BRAND

Semua brand

▼

Wajib Heart Rate Monitor

Wajib GPS

Ringkasan

PRODUK

372

TOTAL DB

378

HARGA RATA-RATA

\$295

HRM tersedia: 100% produk

GPS tersedia: 92% produk

EKSPLORASI DATA

MANAJEMEN DATA

PEMBOBOTAN AHP

HASIL REKOMENDASI

Deploy

Pilih Operasi

Tambah

Lihat Semua

Edit

Hapus

Hapus Produk

Pilih produk:

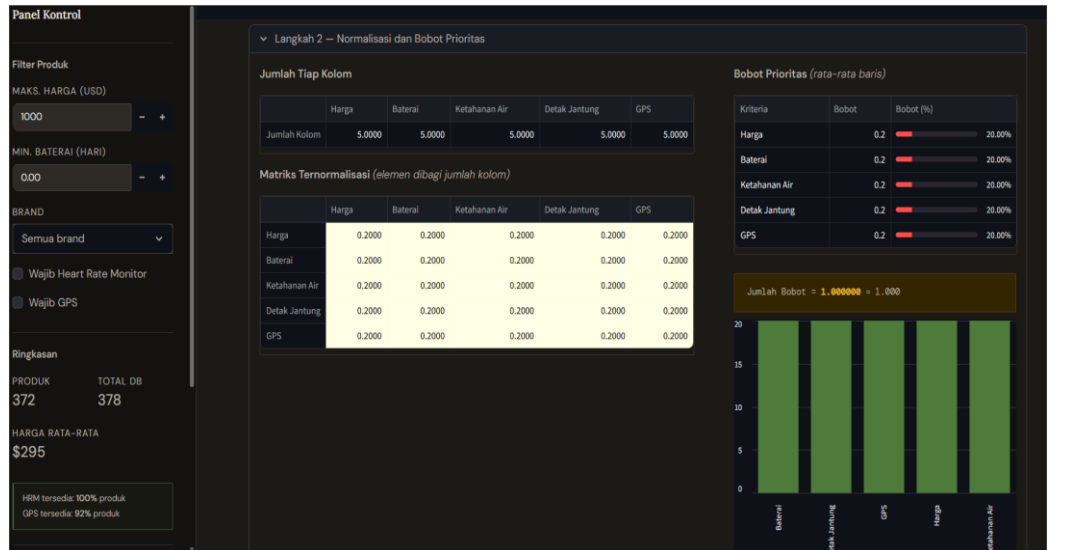
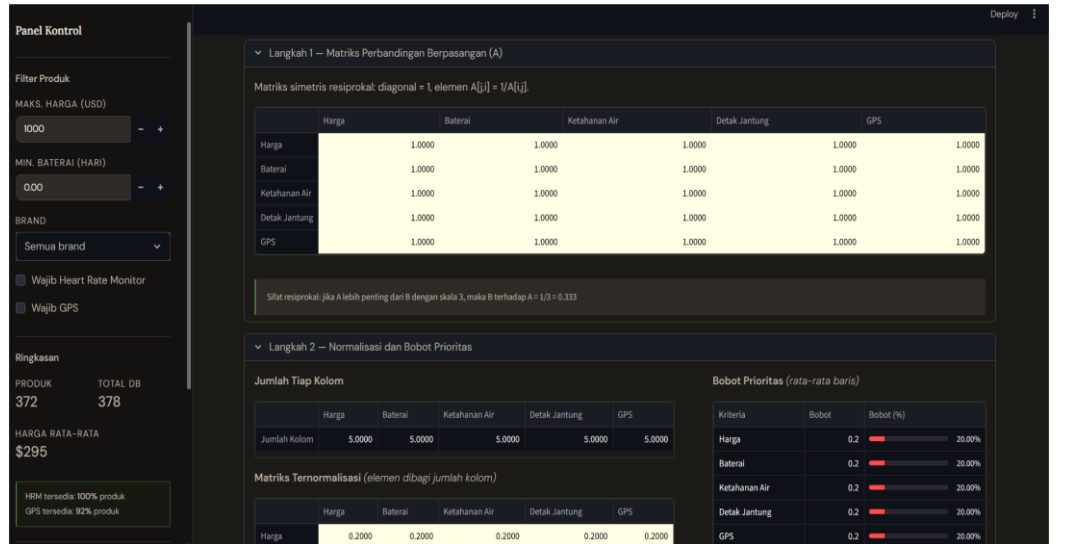
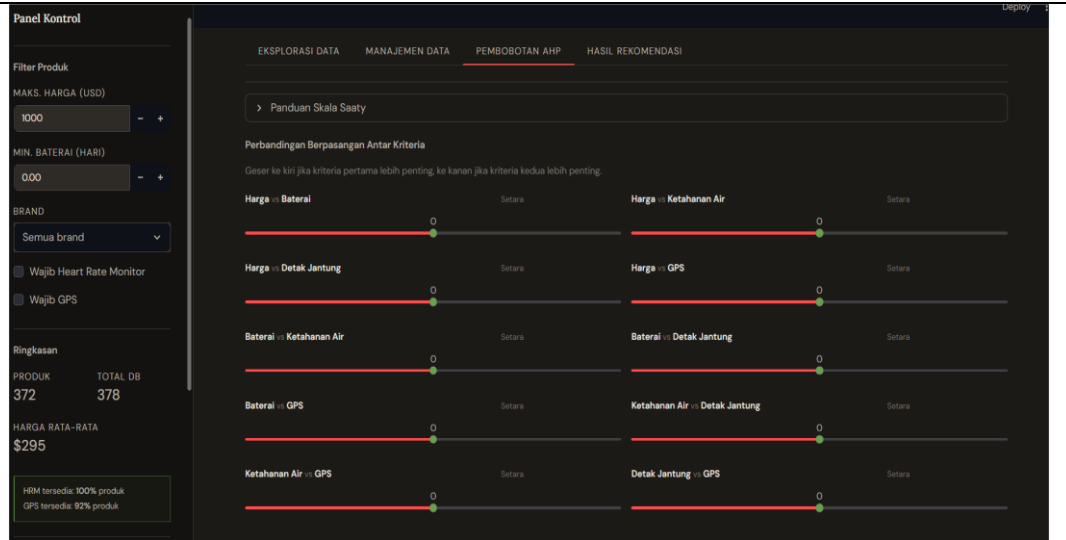
[0] Apple Watch Series 7 — \$399.00

▼

Anda akan menghapus: Apple Watch Series 7 — \$399.00

KONFIRMASI HAPUS

44



Panel Kontrol

Filter Produk

MAKS. HARGA (USD)

1000

MIN. BATERAI (HARI)

0.00

BRAND

Semua brand

Wajib Heart Rate Monitor

Wajib GPS

Ringkasan

PRODUK

372

TOTAL DB

378

HARGA RATA-RATA

\$295

HRM tersedia: 100% produk

GPS tersedia: 92% produk

Langkah 3 — Perhitungan Lambda Maksimum

Kriteria	Vektor Tertimbang (Aw)	Bobot (w)	Rasio (Aw/w)
Harga		1	0.2
Baterai		1	0.2
Ketahanan Air		1	0.2
Detak Jantung		1	0.2
GPS		1	0.2

$$\lambda_{\text{max}} = (5.0000 + 5.0000 + 5.0000 + 5.0000 + 5.0000) / 5$$
$$\lambda_{\text{max}} = 25.000000 / 5$$
$$\lambda_{\text{max}} = 5.000000$$

Semakin dekat λ_{max} dengan n , semakin konsisten matriks. Jika $\lambda_{\text{max}} = n$, maka konsistensi sempurna.

Langkah 4 — CI, RI, dan CR

Tabel Random Index (RI) — Saaty, 1980

n=1	n=2	n=3	n=4	n=5	n=6	n=7	n=8	n=9	n=10
0.0	0.0	0.58	0.9	1.12	1.24	1.32	1.41	1.45	1.49

Kotak terang = nilai yang digunakan: $n = 5$, $RI = 1.12$

Panel Kontrol

Filter Produk

MAKS. HARGA (USD)

1000

MIN. BATERAI (HARI)

0.00

BRAND

Semua brand

Wajib Heart Rate Monitor

Wajib GPS

Ringkasan

PRODUK

372

TOTAL DB

378

HARGA RATA-RATA

\$295

HRM tersedia: 100% produk

GPS tersedia: 92% produk

Consistency Index (CI)

$$CI = (\lambda_{\text{max}} - n) / (n - 1)$$
$$CI = (5.0000 - 5) / (5 - 1)$$
$$CI = 0.000000 / 4$$
$$CI = 0.000000$$

CI = 0 artinya matriks sempurna konsisten.

Random Index (RI)

$n = 5$ kriteria
Lookup tabel Saaty:
 $RI = 1.12$

RI = rata-rata CI dari 500 matriks acak berukuran $n \times n$.

Consistency Ratio (CR)

$CR = CI / RI$ $CR = 0.000000 / 1.12$ $CR = 0.000000 (0.00\%)$

Status Konsistensi

STATUS

Konsisten — $CR = 0.0000 (0.00\%)$ tidak melebihi 10%.

Preferensi penilaian Anda logis dan konsisten. Bobot kriteria ini dapat digunakan secara sah dalam pengambilan keputusan AHP.

Panduan Interpretasi CR:

$CR = 0.00$ — Konsistensi sempurna (ideal, jarang terjadi)
 $CR \leq 0.10$ — Dapat diterima, hasil dapat dipercaya
 $CR > 0.10$ — Tidak konsisten, perlu revisi penilaian

Contoh inkonsistensi: Harga 3x lebih penting dari Baterai, Baterai 3x lebih penting dari GPS, namun Harga hanya 2x lebih penting dari GPS. Ini kontradiksi (seharusnya = 9x).

Panel Kontrol

Filter Produk

MAKS. HARGA (USD)

1000

MIN. BATERAI (HARI)

0.00

BRAND

Semua brand

Wajib Heart Rate Monitor

Wajib GPS

Ringkasan

PRODUK

372

TOTAL DB

378

HARGA RATA-RATA

\$295

HRM tersedia: 100% produk

GPS tersedia: 92% produk

EKSPLOKASI DATA

MANAJEMEN DATA

PEMBOBOTAN AHP

HASIL REKOMENDASI

Podium Rekomendasi

PERINGKAT PERTAMA

Diesel On Fadelite

\$275 • Skor 0.544

PERINGKAT KEDUA

Garmin Enduro

\$799 • Skor 0.544

PERINGKAT KETIGA

Matrix PowerWatch 2

\$499 • Skor 0.544

Top 10 dari 372 Produk

Peringkat	Brand	Model	Harga (USD)	Baterai (hr)	Tahan Air (m)	Heart Rate Monitor	GPS	Skor AHP
1	Diesel	On Fadelite	\$275.00	2.0	30	Ya	Ya	0.544
2	Garmin	Enduro	\$799.00	70.0	100	Ya	Ya	0.544
3	Matrix	PowerWatch 2	\$499.00	0.0	200	Ya	Ya	0.544
4	Garmin	Enduro	\$799.00	70.0	100	Ya	Ya	0.544
5	Casio	G-Shock GSW-H1000	\$699.00	1.0	200	Ya	Ya	0.529
6	TicWatch	Pro 3	\$299.00	72.0	30	Ya	Ya	0.509
7	Samsung	Galaxy Watch Active2	\$249.00	60.0	50	Ya	Ya	0.484
8	Amazfit	Bip S	\$69.00	40.0	50	Ya	Tidak	0.470
9	Xiaomi	Mi Watch Lite	\$49.00	9.0	50	Ya	Ya	0.466
10	Amazfit	GTR 2e	\$139.00	45.0	50	Ya	Ya	0.457

Bobot Digunakan

Kriteria	Bobot (%)
Harga	20.0%
Baterai	20.0%
Ketahanan Air	20.0%
Detak Jantung	20.0%
GPS	20.0%

$CR = 0.0000$

Konsisten

46



BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	2026				
		Minggu				
		1	2	3	4	dst
1	Studi literatur metode AHP dan analisis kebutuhan sistem	✓	✓			
2	Perancangan arsitektur sistem dan desain antarmuka		✓	✓		
3	Implementasi preprocessing data dan algoritma AHP		✓	✓		
4	Pengembangan fitur CRUD dan visualisasi data			✓		
5	Pengujian sistem dan validasi hasil perankingan			✓	✓	
6	Penyusunan laporan proyek akhir				✓	

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Azarya Shalom Widura	123240232	Perancangan arsitektur dan desain antarmuka sistem; Implementasi tab Eksplorasi Data, Pembobotan AHP, dan Hasil Rekomendasi; Pengembangan algoritma preprocessing data dan perhitungan AHP; Penyusunan Bab I dan Bab II laporan
2	Ajrun Ramadhan	123240254	Implementasi fitur CRUD (tab Manajemen Data) dan panel kontrol filter sidebar; Pengujian sistem dan validasi hasil perankingan; Pengumpulan dan analisis dataset Smart_watch_prices.csv; Penyusunan Bab III, Bab IV, dan Daftar Pustaka laporan

BAB IV

KESIMPULAN DAN SARAN

4.2 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian Sistem Pendukung Keputusan Pemilihan Smartwatch menggunakan metode Analytical Hierarchy Process (AHP), dapat ditarik beberapa kesimpulan sebagai berikut:

1. Sistem Pendukung Keputusan berbasis web berhasil dikembangkan menggunakan Python dan Streamlit, mampu memproses dataset berisi 379 produk smartwatch dari berbagai merek global dengan lima kriteria evaluasi utama: harga, ketahanan baterai, ketahanan air, heart rate monitor, dan GPS.
2. Metode AHP berhasil diimplementasikan secara penuh, mulai dari pembangunan matriks perbandingan berpasangan yang bersifat resiprokal, normalisasi matriks, perhitungan bobot prioritas, hingga validasi konsistensi melalui Consistency Ratio (CR). Penilaian dinyatakan valid dan dapat dipercaya apabila nilai $CR \leq 0,10$.
3. Fitur CRUD (Create, Read, Update, Delete) memungkinkan pengguna mengelola data produk secara mandiri dan fleksibel, sehingga sistem tidak terbatas pada dataset awal dan dapat dikembangkan secara dinamis sesuai kebutuhan.
4. Antarmuka web interaktif yang dibangun berhasil menyajikan seluruh tahapan perhitungan AHP secara transparan—mulai dari matriks perbandingan, normalisasi, perhitungan lambda maksimum, hingga validasi CR—sehingga pengguna dapat memahami logika di balik rekomendasi yang dihasilkan.
5. Sistem terbukti memberikan rekomendasi yang berbeda-beda tergantung pada preferensi bobot yang dikonfigurasi pengguna, yang menunjukkan bahwa sistem bekerja secara adaptif dan personalisasi sesuai dengan kebutuhan individu.

4.3 Saran

Untuk pengembangan sistem ke depannya, beberapa saran yang dapat dijadikan pertimbangan adalah sebagai berikut:

1. Integrasi lebih banyak kriteria evaluasi seperti resolusi layar, konektivitas (NFC, LTE), tipe display, dan OS kompatibilitas untuk menghasilkan rekomendasi yang lebih komprehensif dan menyeluruh.
2. Penambahan fitur pengguna agar setiap pengguna dapat menyimpan konfigurasi bobot kriteria yang telah dibuat, sehingga tidak perlu mengulang proses pembobotan setiap kali menggunakan sistem.
3. Penambahan metode SPK lainnya seperti TOPSIS, SAW, atau VIKOR sebagai pembanding untuk menghasilkan rekomendasi yang lebih robust dan memberikan perspektif multi-metode.
4. Integrasi data real-time dari platform e-commerce seperti Tokopedia atau Shopee menggunakan web scraping atau API, sehingga data harga yang digunakan selalu terbaru dan mencerminkan kondisi pasar saat ini.
5. Pengembangan fitur penjelasan rekomendasi yang lebih detail, misalnya menampilkan alasan mengapa suatu produk menempati peringkat tertentu dalam bentuk narasi atau highlight kriteria unggulan setiap produk.

DAFTAR PUSTAKA

- Kusrini. (2007). *Konsep dan Aplikasi Sistem Pendukung Keputusan*. Yogyakarta: Penerbit Andi.
- Saaty, T. L. (1980). *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. New York: McGraw-Hill.
- Saaty, T. L. (2008). Decision Making with the Analytic Hierarchy Process. *International Journal of Services Sciences*, 1(1), 83–98.
- Turban, E., Aronson, J. E., & Liang, T. P. (2005). *Decision Support Systems and Intelligent Systems* (7th ed.). Upper Saddle River, NJ: Pearson Prentice Hall.
- Streamlit Inc. (2023). *Streamlit Documentation*. Diakses dari <https://docs.streamlit.io>
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). Sebastopol, CA: O'Reilly Media.
- Harris, C. R., et al. (2020). Array Programming with NumPy. *Nature*, 585(7825), 357–362.